

■ Lab 15 — Advanced Terraform & CI/CD avec GitHub Actions

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris et utilisé le **dependency lock file** (`terraform.lock.hcl`)
- Renommé une ressource avec le bloc `moved` sans la détruire/recréer
- Validé l'infrastructure avec le bloc `check` (assertions post-apply)
- Géré les **secrets AWS** de manière sécurisée via les variables CI/CD
- Utilisé `terraform plan -out=planfile + terraform apply planfile`
- Compris et résolu un problème de **dépendance de destruction** avec `depends_on`
- Mis en place un pipeline **CI/CD complet** avec GitHub Actions

■ Étape 1 — Créer le `.gitignore` AVANT tout

■ ***Cette étape est critique — à faire en premier avant tout `git add`.***

```
cd labs/lab15-advanced
```

```
cat > .gitignore << 'EOF'
# Dossier providers Terraform (très lourd ~700MB - jamais commiter)
.terraform/

# Plan files (contiennent des secrets - jamais commiter)
tfplan
tfplan-destroy
*.tfplan

# State local
terraform.tfstate
terraform.tfstate.backup

# Variables sensibles
*.auto.tfvars
EOF
```

■ *Le dossier `.terraform/` contient les binaires des providers (~700MB pour AWS) — GitHub bloque les fichiers > 100MB.*

Le fichier `tfplan` contient des credentials en clair — ne jamais le commiter.

■ Étape 2 — Créer les fichiers Terraform

`backend.tf`

```
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region     = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

■ ■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}

variable "environment" {
  description = "Environnement de déploiement"
  type        = string
  default     = "dev"

  validation {
    condition     = contains(["dev", "prod"], var.environment)
    error_message = "L'environnement doit être dev ou prod."
  }
}
```

`locals.tf`

```
locals {
  prefix = "lab15-${var.username}-${var.environment}"

  common_tags = {
    Lab           = "lab15"
    Username      = var.username
    Environment   = var.environment
    ManagedBy     = "Terraform"
  }
}
```

`main.tf` — Version initiale (sans `depends\_on`)

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "lab15_sg" {
  name          = "${local.prefix}-sg"
  description    = "Security group Lab 15 - ${var.username}"

  ingress {
    from_port     = 80
    to_port       = 80
    protocol       = "tcp"
    cidr_blocks    = ["0.0.0.0/0"]
    description    = "HTTP"
  }

  egress {
    from_port     = 0
    to_port       = 0
    protocol       = "-1"
    cidr_blocks    = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}

resource "aws_instance" "lab15_instance" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab15_sg.id]
  associate_public_ip_address = true

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}
```

outputs.tf

```
output "instance_id" {
  description = "ID de l'instance"
  value      = aws_instance.lab15_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab15_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab15_sg.id
}
```

■ Étape 3 — Dependency Lock File (`terraform.lock.hcl`)

```
terraform init
cat .terraform.lock.hcl
```

■ Le fichier `.terraform.lock.hcl` **verrouille les versions exactes** des providers.

Il doit être **commité dans Git** — contrairement au dossier `.terraform/` qui lui ne doit jamais l'être.

■ Étape 4 — Plan file : `terraform plan -out`

```
# Générer le plan dans un fichier
terraform plan -var="username=<votre-prenom>" -out=tfplan

# Inspecter le plan
terraform show tfplan

# Appliquer exactement ce plan
terraform apply tfplan
```

■ Le fichier `tfplan` est dans le `.gitignore` — il ne sera jamais commité.

C'est le pattern CI/CD standard : `plan` dans la PR, `apply` après merge.

■ Étape 5 — Découvrir le problème `depends\_on`

Étape 5.1 — Lancer le destroy sans `depends\_on`

```
terraform destroy -var="username=<votre-prenom>"
```

Observez ce qui se passe — après quelques minutes vous verrez :

```
aws_security_group.lab15_sg: Still destroying... [id=sg-xxx, 1m00s elapsed]
aws_security_group.lab15_sg: Still destroying... [id=sg-xxx, 2m00s elapsed]
aws_security_group.lab15_sg: Still destroying... [id=sg-xxx, 3m00s elapsed]
...
```

■ **Pourquoi ?** Terraform détruit les ressources **en parallèle** par défaut. Il essaie de supprimer l'instance EC2 ET le Security Group en même temps. Mais AWS refuse de supprimer un Security Group tant qu'une instance y est encore attachée — même si cette instance est en cours de suppression.

■ **Résultat :** Terraform attend indéfiniment qu'AWS libère le Security Group, ce qui peut prendre très longtemps.

Étape 5.2 — Débloquer la situation

Si vous êtes bloqué, appuyez sur **Ctrl+C** puis terminez l'instance manuellement :

```
INSTANCE_ID=$(terraform state show aws_instance.lab15_instance | grep '"id"' | head -1 | awk -F'"' '{pri
aws ec2 terminate-instances --instance-ids $INSTANCE_ID

# Attendre que l'instance soit terminée
aws ec2 wait instance-terminated --instance-ids $INSTANCE_ID

# Relancer le destroy
terraform destroy -var="username=<votre-prenom>"
```

■ Étape 6 — Solution : `depends\_on`

Étape 6.1 — Comprendre `depends\_on`

Terraform gère les dépendances **implicites** automatiquement quand une ressource référence une autre (`aws_instance` référence `aws_security_group.lab15_sg.id` → Terraform sait créer le SG avant l'instance).

Mais pour la **destruction**, Terraform ne connaît pas toujours l'ordre correct. C'est là qu'intervient `depends_on` — une **dépendance explicite**.

Étape 6.2 — Corriger `main.tf` avec `depends_on`

Mettez à jour le bloc `aws_security_group` dans `main.tf` :

```
resource "aws_security_group" "lab15_sg" {
  name           = "${local.prefix}-sg"
  description    = "Security group Lab 15 - ${var.username}"

  ingress {
    from_port     = 80
    to_port       = 80
    protocol      = "tcp"
    cidr_blocks   = ["0.0.0.0/0"]
    description   = "HTTP"
  }

  egress {
    from_port     = 0
    to_port       = 0
    protocol      = "-1"
    cidr_blocks   = ["0.0.0.0/0"]
  }

  # depends_on force Terraform à détruire l'instance AVANT le Security Group
  depends_on = [aws_instance.lab15_instance]

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}
```

■ `depends_on = [aws_instance.lab15_instance]` dit à Terraform :

- **Création** : créer le SG avant l'instance (ordre normal déjà respecté)

- **Destruction** : détruire l'instance AVANT de supprimer le SG ■

Étape 6.3 — Redéployer et tester le destroy

```
terraform apply -var="username=<votre-prenom>"
```

Maintenant lancez le destroy :

```
terraform destroy -var="username=<votre-prenom>"
```

Observez la différence — Terraform détruit maintenant l'instance EN PREMIER :

```
aws_instance.lab15_instance: Destroying...  
aws_instance.lab15_instance: Still destroying... [10s elapsed]  
aws_instance.lab15_instance: Destruction complete after 32s  
aws_security_group.lab15_sg: Destroying... ← seulement après l'instance  
aws_security_group.lab15_sg: Destruction complete after 1s ■
```

■ Étape 7 — Bloc `moved` (renommer sans recréer)

Dans `main.tf`, renommez `lab15_instance` en `web_server` :

```
resource "aws_instance" "web_server" {    # ← renommé  
  ami                = data.aws_ami.amazon_linux.id  
  instance_type      = "t2.micro"  
  vpc_security_group_ids = [aws_security_group.lab15_sg.id]  
  associate_public_ip_address = true  
  
  tags = merge(local.common_tags, {  
    Name = "${local.prefix}-instance"  
  })  
}
```

Mettez à jour le `depends_on` dans `aws_security_group` :

```
depends_on = [aws_instance.web_server]    # ← nom mis à jour
```

Mettez à jour `outputs.tf` :

```
output "instance_id" {  
  value = aws_instance.web_server.id  
}  
  
output "instance_public_ip" {  
  value = aws_instance.web_server.public_ip  
}
```

Créez `moved.tf` :

```
moved {  
  from = aws_instance.lab15_instance  
  to   = aws_instance.web_server  
}
```

```
terraform plan -var="username=<votre-prenom>"
```


Résultat attendu :

```
# aws_instance.lab15_instance has moved to aws_instance.web_server
Plan: 0 to add, 0 to change, 0 to destroy.
```

```
terraform apply -var="username=<votre-prenom>"
```

■ Étape 8 — Bloc `check` (assertions post-apply)

Ajoutez dans `main.tf` :

```
check "instance_is_running" {
  data "aws_instance" "verify" {
    instance_id = aws_instance.web_server.id
  }

  assert {
    condition     = data.aws_instance.verify.instance_state == "running"
    error_message = "L'instance ${aws_instance.web_server.id} n'est pas en état running !"
  }
}

check "security_group_exists" {
  data "aws_security_group" "verify" {
    id = aws_security_group.lab15_sg.id
  }

  assert {
    condition     = data.aws_security_group.verify.id != ""
    error_message = "Le Security Group n'existe pas !"
  }
}
```

```
terraform apply -var="username=<votre-prenom>"
```

■ Le bloc `check` s'exécute **après** le déploiement et vérifie l'état réel. Un échec affiche un **warning** mais n'arrête pas l'apply.

■ Étape 9 — Gestion des secrets (bonnes pratiques)

■ **Règle absolue** : ne jamais mettre de secrets dans le code ou `terraform.tfvars`.

```
# ■ INTERDIT
variable "aws_access_key" {
  default = "AKIAXXXXXXXXXXXXXXXXXX"
}
```

```
# ■ Variables d'environnement localement
export AWS_ACCESS_KEY_ID="AKIAXXXXXXXXXXXXXXXXXX"
export AWS_SECRET_ACCESS_KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

En CI/CD, les secrets sont injectés via **GitHub Actions Secrets** — jamais en clair dans le code.

■ Étape 10 — CI/CD avec GitHub Actions

Étape 10.1 — Configurer les secrets GitHub

Dans votre repo GitHub :

1. **Settings** → **Secrets and variables** → **Actions**

2. **New repository secret** — ajoutez :

- `AWS_ACCESS_KEY_ID`

- `AWS_SECRET_ACCESS_KEY`

Étape 10.2 — Créer le workflow à la racine du repo

■■ Le dossier `.github/workflows/` doit être à la ****racine du repo****, pas dans `labs/lab15-advanced/`.

```
# Se placer à la racine du repo
cd /workspaces/tf-training-<votre-prenom>

mkdir -p .github/workflows
```

Créez `.github/workflows/terraform.yml` :

```
name: Terraform CI/CD

on:
  push:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  pull_request:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  workflow_dispatch:
    inputs:
      action:
        description: 'Action à exécuter'
        required: true
        default: 'plan'
        type: choice
        options: [ plan, apply, destroy ]

env:
  TF_WORKING_DIR: labs/lab15-advanced
  TF_VAR_username: ${ github.actor }

jobs:
  terraform:
    name: Terraform ${ github.event.inputs.action || 'plan' }
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ${ env.TF_WORKING_DIR }
```

steps:

- name: Checkout code
uses: actions/checkout@v4
- name: Setup Terraform
uses: hashicorp/setup-terraform@v3
with:
 terraform_version: "1.15.5"
- name: Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v4
with:
 aws-access-key-id: \${ secrets.AWS_ACCESS_KEY_ID }
 aws-secret-access-key: \${ secrets.AWS_SECRET_ACCESS_KEY }
 aws-region: eu-west-1
- name: Terraform Init
run: terraform init
- name: Terraform Format Check
run: terraform fmt -check -recursive
- name: Terraform Validate
run: terraform validate
- name: Terraform Plan
run: terraform plan -var="environment=dev" -out=tfplan
- name: Terraform Apply

run: terraform apply tfplan
if: github.event.inputs.action == 'apply'
- name: Terraform Destroy
run: |
 terraform plan -destroy -var="environment=dev" -out=tfplan-destroy
 terraform apply tfplan-destroy
if: github.event.inputs.action == 'destroy'

■ ****Tout dans un seul job**** — `plan` et `apply` s'exécutent dans le même environnement.

Si `plan` et `apply` étaient dans des jobs séparés, le fichier `tfplan` serait introuvable dans le job `apply` car chaque job repart d'un environnement vierge.

■■ ****L'apply n'est jamais automatique**** — un push sur `main` déclenche uniquement le `plan`. Pour appliquer ou détruire, il faut toujours passer par `workflow\_dispatch` et choisir explicitement l'action. Cela évite toute destruction accidentelle de l'infrastructure.

Étape 10.3 — Vérifier et commiter

```
cd /workspaces/tf-training-<votre-prenom>

# Vérifier ce qui sera commité
git status
# ■ .github/workflows/terraform.yml
# ■ labs/lab15-advanced/.gitignore
# ■ labs/lab15-advanced/.terraform.lock.hcl
# ■ labs/lab15-advanced/*.tf
# ■ NE PAS voir : .terraform/, tfplan

git add .
git commit -m "feat: lab15 advanced terraform + CI/CD"
git push origin main
```

■ Étape 11 — Tester le workflow

Scénario 1 — Plan sur Push

```
git add . && git commit -m "test: declencher pipeline" && git push origin main
```

■ Le pipeline exécute uniquement `terraform plan` — aucune ressource n'est créée ou détruite automatiquement.

Scénario 2 — Apply manuel (créer / mettre à jour)

1. GitHub → **Actions** → **Terraform CI/CD**
2. **Run workflow** → sélectionnez `apply` → **Run workflow**

■ Le pipeline exécute `plan` puis `apply` — les ressources sont créées ou mises à jour.

Scénario 3 — Destroy manuel (détruire)

1. GitHub → **Actions** → **Terraform CI/CD**
2. **Run workflow** → sélectionnez `destroy` → **Run workflow**

■ Le pipeline exécute `plan -destroy` puis `apply` — toutes les ressources sont détruites.

■■ Cette action est irréversible — à utiliser uniquement en fin de lab.

■ Étape 12 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"  
rm -f tfplan tfplan-destroy
```

■ Validation du Lab

#	Critère	Validé
1	<code>.gitignore</code> créé avant le premier <code>git add</code>	■
2	<code>.terraform/</code> et <code>tfplan</code> absents du repo GitHub	■
3	<code>.terraform.lock.hcl</code> présent et commité	■
4	<code>terraform plan -out=tfplan</code> + <code>terraform apply tfplan</code> fonctionne	■
5	Problème de destroy sans <code>depends_on</code> observé	■
6	<code>depends_on</code> résout le problème — instance détruite avant le SG	■
7	Bloc <code>moved</code> renomme sans recréer (<code>0 to destroy</code>)	■
8	Bloc <code>check</code> valide l'état après apply	■
9	Secrets AWS dans GitHub Actions Secrets (jamais dans le code)	■
10	<code>.github/workflows/</code> à la racine du repo	■
11	Pipeline GitHub Actions se déclenche sur push	■
12	Apply et Destroy via <code>workflow_dispatch</code> fonctionnent	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
SG bloqué en destroy > 5 min	Instance encore attachée au SG	Ajouter <code>`depends_on = [aws_instance.web_server]`</code> dans le SG
<code>`File exceeds 100MB`</code>	<code>`.terraform/`</code> commité	Ajouter <code>`.gitignore`</code> avant <code>`git add`</code>
<code>`stat tfplan: no such file or directory`</code>	Plan et Apply dans des jobs séparés	Utiliser un seul job dans le workflow
Pipeline introuvable dans Actions	<code>`.github/`</code> dans le mauvais dossier	Placer <code>`.github/workflows/`</code> à la racine du repo
<code>`fmt -check`</code> échoue	Code mal formaté	Lancer <code>`terraform fmt -recursive`</code> localement
<code>`moved`</code> recrée la ressource	Mauvais nom dans <code>`from`</code>	Vérifier avec <code>`terraform state list`</code>

■ ****Fin du Lab 15 — Vous maîtrisez les bonnes pratiques avancées et le CI/CD Terraform !****

*Félicitations — vous avez complété les ****15 labs**** de la formation Terraform ! ■*